

Keystroke Continuous Authentication — Team Implementation Plan

Team: You (ML-strong) + Friend (ML-learning) **Split logic:** You own the ML core (model design, feature science, evaluation). Friend owns the software engineering layer (data pipeline, integration, UI) — with a few guided ML tasks so he actually learns, not just codes plumbing.

Role Split at a Glance

Area	Owner	Why
Feature engineering design (which features, why)	You	Needs ML intuition — deciding what signal matters
Model selection, training, tuning	You	Core ML skill, needs sklearn depth
Evaluation (FAR/FRR, ROC, threshold tuning)	You	Needs stats background
Sliding window + decision logic	Shared	You design the logic, friend implements it in code — good learning bridge
Data collector (Phase 1 script)	Friend	Mostly software engineering (pynput, file I/O), not ML-heavy
Feature extraction <i>code</i> (not design)	Friend, reviewed by you	You specify the formulas, he writes the pandas code — real learning task
Background service + startup integration	Friend	OS/systems work, no ML needed
Notifications/lock-screen trigger	Friend	Pure software engineering
System tray UI (optional)	Friend	Good beginner-friendly scope-expansion

polish)		task
Write-up / research framing	Shared	You draft the ML/research sections, friend documents the system architecture

Phase-by-Phase Breakdown

Phase 1 — Data Collection (Week 1)

- **Friend builds:** the `collector.py` script (pynput listener, CSV logging) — he already has the fully-explained reference version, so this is a great low-risk starting task to build confidence.
- **You do:** review his code, sanity-check that timestamps/dwell/flight math will come out clean, decide the target text/prompts for consistent baseline data collection.
- **Both:** each collect your own 1500-2000 keystroke baseline (you need your own data as the "authorized user" baseline; his data becomes your "imposter" test set later).

Phase 2 — Feature Engineering (Week 1-2)

- **You do:** decide the exact feature list and formulas (dwell, flight, digraphs, typing speed, backspace rate) and write this up as a spec (like a mini design doc) — this is the ML judgment part.
- **Friend does:** implement the spec in pandas — turning your formulas into a function that converts raw CSV into windowed feature vectors. This is his best learning opportunity: he's translating math into code with your formulas as a safety net.
- **Checkpoint:** you review his feature extraction output on a small sample before moving forward — catch bugs early (e.g., negative flight times, backspace key IDs skewing stats).

Phase 3 — Model Training & Evaluation (Week 2)

- **You own this entirely:** Isolation Forest / One-Class SVM training, threshold selection, FAR/FRR calculation, ROC curve.
- **Friend's role:** shadow you here — walk him through *why* you pick a threshold, what FAR/FRR mean. This is a good teaching checkpoint rather than a coding task, since model tuning needs the ML judgment you already have.
- **Test data:** use his baseline as your "imposter" data and vice versa (swap datasets)

to validate the model actually discriminates between you two.

Phase 4 — Sliding Window + Decision Logic (Week 3)

- **You design:** the window size, slide interval, and the "N out of last 5 windows" alert rule (the logic we worked through earlier).
- **Friend implements:** the actual `deque`-based buffer and threshold-counting code, based on your spec — another good code-writing task with clear, well-defined ML-adjacent logic to work from.

Phase 5 — Real-Time Integration (Week 3-4)

- **Friend owns this fully:** background service setup, Task Scheduler/startup config, `plyer / win10toast` notifications, `LockWorkStation()` trigger, and (optional) `pystray` tray icon. This is the biggest chunk of pure software engineering — a good way for him to own a full deliverable end-to-end.
- **You do:** integrate your trained model (`model.pkl`) into his service — make sure `joblib.load()` and your scoring function plug cleanly into his real-time loop.

Phase 6 — Evaluation & Write-Up (Week 4)

- **You write:** the ML/research sections — methodology, FAR/FRR results, drift analysis (does the model hold up over days/different keyboards), and the "why free-text continuous auth is a harder research problem" framing for resume/research value.
- **Friend writes:** the system architecture section — how the pieces fit together, the real-time pipeline, and setup/reproduction instructions.
- **Both:** final testing together — one of you tries to "impersonate" the other's typing deliberately to stress-test the system.

Learning Checkpoints for Friend (so he actually grows in ML)

1. After Phase 2: he should be able to explain what dwell time and flight time are and why windows (not single keystrokes) are the ML input.
2. After Phase 3: he should understand what FAR/FRR mean and why threshold choice is a trade-off, even if you did the tuning.
3. After Phase 4: he should understand why a single anomalous window isn't enough to

trigger an alert (the smoothing/majority-vote logic).

Suggested Meeting Cadence

- Short sync after each phase (not daily) — review what got built, catch issues, hand off the next phase's spec.
- You write a 3-5 line "spec" before each phase where you own the design (Phases 2-4) so friend isn't guessing what to build.